

Exercise-1—Design Patterns and Principles

Implementing the Singleton Pattern

Source code:

```
using System;
public class Computer
{
    private string cpu;
    private string ram = "Not specified";
    private string storage = "Not specified";
    private string gpu = "Not specified";
    private string motherboard = "Not specified";
    private string powerSupply = "Not specified";
    private string coolingSystem = "Not specified";
    private bool wifi = false;
    private bool bluetooth = false;
    private Computer() { }
    public override string ToString()
    {
        return $"CPU: {cpu}\nRAM: {ram}\nStorage: {storage}\nGPU:
{gpu}\nMotherboard: {motherboard}\nPower Supply: {powerSupply}\nCooling
System: {coolingSystem}\nWiFi: {wifi}\nBluetooth: {bluetooth}";
    }
    public class Builder
    {
        private Computer computer = new Computer();
        public Builder(string cpu)
        {
            if (string.IsNullOrEmpty(cpu))
                throw new ArgumentException("CPU cannot be empty.");
```

```
        computer.cpu = cpu;
    }
    public Builder Ram(string ram)
    {
        computer.ram = ram;
        return this;
    }
    public Builder Storage(string storage)
    {
        computer.storage = storage;
        return this;
    }
    public Builder Gpu(string gpu)
    {
        computer.gpu = gpu;
        return this;
    }
    public Builder Motherboard(string motherboard)
    {
        computer.motherboard = motherboard;
        return this;
    }
    public Builder PowerSupply(string powerSupply)
    {
        computer.powerSupply = powerSupply;
        return this;
    }
    public Builder CoolingSystem(string coolingSystem)
    {
        computer.coolingSystem = coolingSystem;
        return this;
    }
}
```

```

    }
    public Builder Wifi(bool wifi)
    {
        computer.wifi = wifi;
        return this;
    }
    public Builder Bluetooth(bool bluetooth)
    {
        computer.bluetooth = bluetooth;
        return this;
    }
    public Computer Build()
    {
        return computer;
    }
}

public class Program
{
    public static void Main()
    {
        Console.WriteLine("[1] Basic Office PC");
        Computer officePc = new Computer.Builder("Intel Core i3-13100")
            .Ram("8GB DDR4")
            .Storage("256GB SSD")
            .Motherboard("ASUS Prime H610M")
            .PowerSupply("450W Bronze PSU")
            .Wifi(true)
            .Build();
        Console.WriteLine(officePc);
    }
}

```

```
        Console.WriteLine("\n[2] High-End Gaming Rig");
        Computer gamingRig = new Computer.Builder("AMD Ryzen 9 7950X")
            .Ram("64GB DDR5-6000")
            .Storage("4TB NVMe Gen5 SSD")
            .Gpu("NVIDIA RTX 4090 24GB")
            .Motherboard("ASUS ROG Crosshair X670E")
            .PowerSupply("1000W Platinum PSU")
            .CoolingSystem("360mm AIO Liquid Cooler")
            .Wifi(true)
            .Bluetooth(true)
            .Build();
        Console.WriteLine(gamingRig);
    }
}
```

OUTPUT:

```
[1] Basic Office PC
CPU: Intel Core i3-13100
RAM: 8GB DDR4
Storage: 256GB SSD
GPU: Not specified
Motherboard: ASUS Prime H610M
Power Supply: 450W Bronze PSU
Cooling System: Not specified
WiFi: True
Bluetooth: False

[2] High-End Gaming Rig
CPU: AMD Ryzen 9 7950X
RAM: 64GB DDR5-6000
Storage: 4TB NVMe Gen5 SSD
GPU: NVIDIA RTX 4090 24GB
Motherboard: ASUS ROG Crosshair X670E
Power Supply: 1000W Platinum PSU
Cooling System: 360mm AIO Liquid Cooler
WiFi: True
Bluetooth: True
```